

SALESTORM

The Flash-Sale Integrity Problem

Team :

Fazmina S

Dhakhatha S

Harish k

CONTENTS

01 Who Gets the 100 Products? ..

02 How Do We Stop the 101st? ..

03 Last-Item Race Condition ..

04 10,000 Simultaneous Requests ..

05 50× Traffic Surge ..

06 Payment Succeeds, Order Crashes ..

07 What If Payment Fails? ..

08 Duplicate Click Handling ..

09 Reservation Expiration ..

10 Monitoring & Recovery ..

How Do We Stop the 101st Customer?

Flash sales create extreme concurrency: thousands of customers racing for limited inventory in milliseconds. Without a single source of truth, distributed systems risk overselling—accepting more orders than available stock. The solution requires an authoritative inventory store that enforces one unbreakable rule.

The Core Invariant

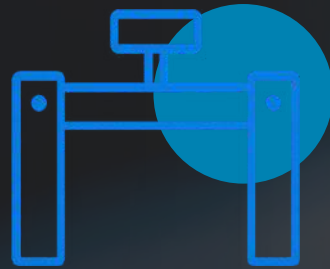
Committed inventory $\leq$ sellable inventory. This invariant is enforced atomically at the authoritative store. Every reservation request is validated against current available count before acceptance. No reservation succeeds if it would violate this constraint—making overselling mathematically impossible.

Recoverable Outcomes

Every transaction reaches a deterministic end state: success, rejection, or compensation. Failed payments trigger automatic inventory release. Expired reservations return stock to the available pool. No inventory gets permanently stuck—the system self-heals through defined recovery paths.



Admission Control



Admission control evaluates each incoming request against current capacity. Only requests that pass validation proceed to the reservation layer. This prevents the system from being overwhelmed and ensures fair, orderly processing.



Authoritative Reservation



The authoritative inventory store maintains the single source of truth. Reservations are granted atomically — each decrement is a transaction. Winners are determined by arrival order at this layer, guaranteeing exactly 100 products go to exactly 100 customers.

Who Gets the 100 Products?

Admission control acts as a gatekeeper, only forwarding requests that can be safely evaluated. The authoritative inventory store then processes reservations atomically. The first 100 valid reservation requests win — each granted a confirmed slot. Request 101 and beyond receive an immediate, deterministic sold-out response. No ambiguity, no race conditions.

Customer A

Requests last item at $T=0.001s$. Transaction enters authoritative inventory store first. Atomic check: $quantity \geq 1$? TRUE. Reserve succeeds. Inventory decrements to 0. Order confirmed.

The serialized transaction model guarantees deterministic ordering. Whichever request the authoritative store processes first wins—the other receives an immediate, definitive rejection.

Customer B

Requests last item at $T=0.002s$. Transaction queued behind Customer A. Atomic check: $quantity \geq 1$? FALSE. Reserve rejected immediately. Clean "sold out" response. No ambiguous state.

No race condition exists because there is no intermediate state. The atomic transaction either reserves or rejects—never both, never partial, never two winners for one item.

Payment Succeeds, Order Service Crashes

Stage 1

Payment authorization succeeds. Before returning to the customer, the transaction outcome is durably recorded in the authoritative store. This creates a recoverable checkpoint — the system knows payment was captured even if downstream services fail.

Checkpoint

The durable transaction log acts as the single source of truth. Payment state is persisted atomically before any response. If order-service crashes mid-flight, the payment record survives and guides recovery.

Stage 2

Reconciliation process detects the missing order acknowledgement by comparing payment records against order confirmations. It safely resumes order creation using the idempotency key — no double-charge, no lost sale.

Recovery

Background reconciler runs periodically, identifying orphaned payments. It replays the order-creation step idempotently — the customer receives their confirmation, inventory remains correctly allocated, and no manual intervention is required.

What If Payment Fails?

Payment failure is not a silent dead-end. The system treats failed payment as a trigger for compensation logic: the reserved inventory is released back to the sellable pool, the reservation record transitions to a terminal "failed" state, and the customer receives an explicit failure notification. No inventory gets stuck in limbo.

Outcome A: Retry Window

If policy allows retries, the reservation remains held for a configured grace period. The customer can attempt payment again within this window. If all retries exhaust or the window expires, the system automatically triggers compensation and releases inventory.

Outcome B: Immediate Release

For hard payment failures (declined card, fraud flag), compensation executes immediately. The authoritative inventory store increments available count atomically, the reservation is marked terminal, and a failure event publishes to the message broker for downstream notification.

Idempotency Key

First Request Creates Result

The idempotency key is stored with the operation outcome. Whether reservation, payment initiation, or order creation—the result is persisted atomically with the key.

Replay Protection

Duplicates Return Same Result

Subsequent requests with the same key skip processing entirely. The stored result is returned immediately—no second reservation, no double charge, no duplicate order.

What If the Same Customer Clicks Buy Now Twice?

API idempotency ensures duplicate requests never create duplicate reservations, payments, or orders. The client generates a unique idempotency key per logical purchase attempt. The system returns the original result for any repeated request.





Happy Path

Reservation Lifecycle

Available → Reserved → Paid/Committed

Each reservation carries a TTL. If payment completes within the window, inventory transitions to committed. The atomic state machine prevents any ambiguity.



Timeout Path

Expiry & Late Payment

Reserved → Expired → Released

When TTL elapses without payment, inventory returns to the available pool. Late-arriving payments trigger reconciliation logic, not blind attachment.



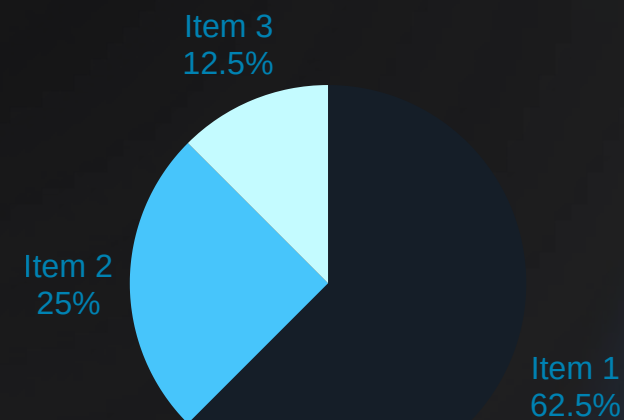
What Happens When a Reservation Expires?

Late payments are never blindly attached. The system reconciles against current reservation state — if the item was released, the payment is automatically refunded or flagged for manual review. No orphaned payments, no inventory corruption.

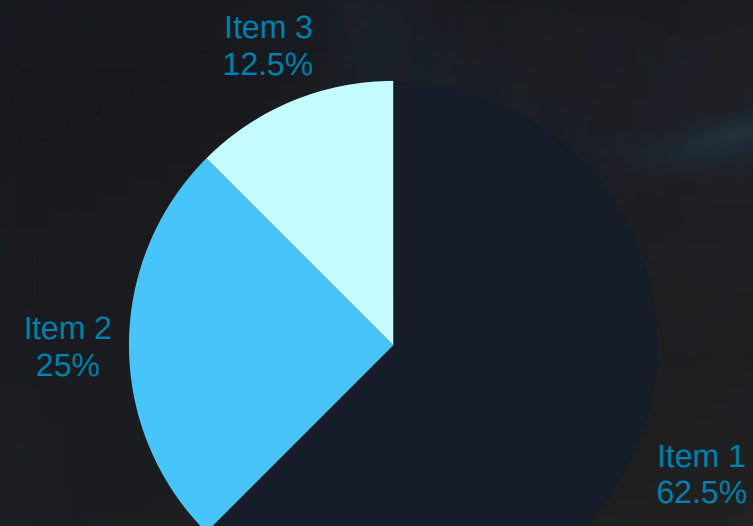
How Do We Handle 10,000 Simultaneous Requests?

A layered traffic-control funnel ensures controlled degradation rather than catastrophic failure. Edge admission control rejects or defers requests beyond capacity thresholds. Accepted traffic enters bounded queues that smooth burst patterns. The message broker decouples ingestion from processing, preventing origin overload. Worker pools with fixed concurrency protect the authoritative inventory store from stampede. Every rejected request receives an explicit response—no silent failures or timeouts.

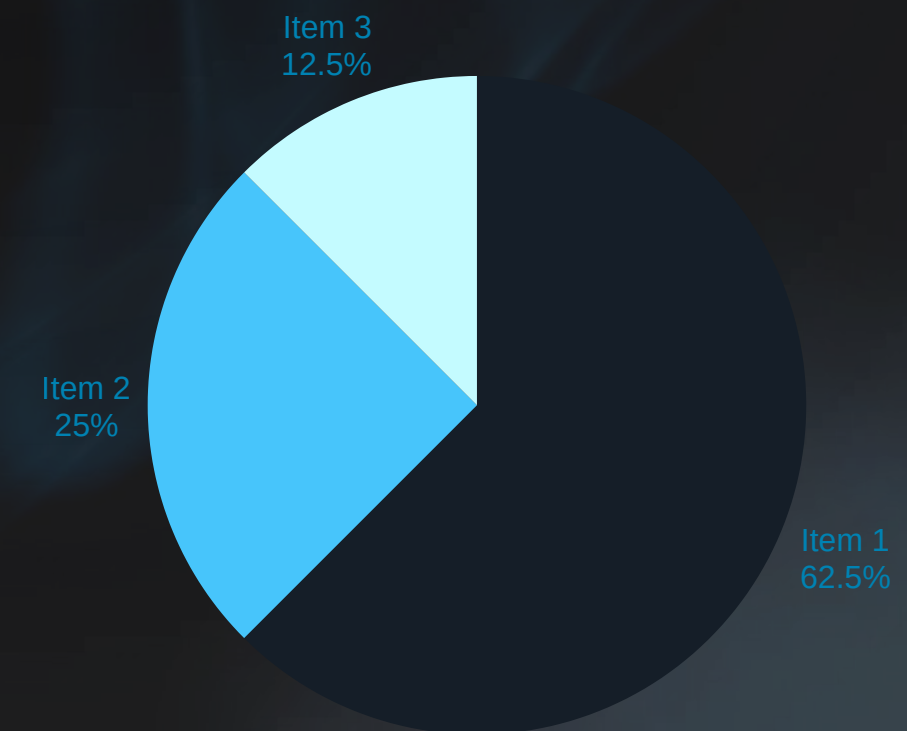
Traffic Reduction by Layer



Request Flow Through Layers



Worker Utilization



50× Traffic Surge

The system absorbs demand at the edge through CDN caching and rate limiting. Only qualified requests pass admission control to reach the authoritative store. Queue depth bounds prevent origin overload.

Graceful degradation activates automatically: non-critical features shed load first, reservation timeouts shorten, and the admission controller tightens thresholds. The invariant—never oversell—remains intact regardless of traffic volume.

99%

Edge Absorption

Edge absorption ratio at peak load. CDN and admission control filter 99% of requests before they reach the authoritative transaction path.

1,000

Origin Bound

Maximum concurrent requests the authoritative store processes, regardless of inbound volume. Bounded queues ensure predictable latency.

**Telemetry Collection**

Streams reservation, payment, queue depth metrics

**Anomaly Detection**

Triggers alerts on threshold breaches

**Recovery Workers**

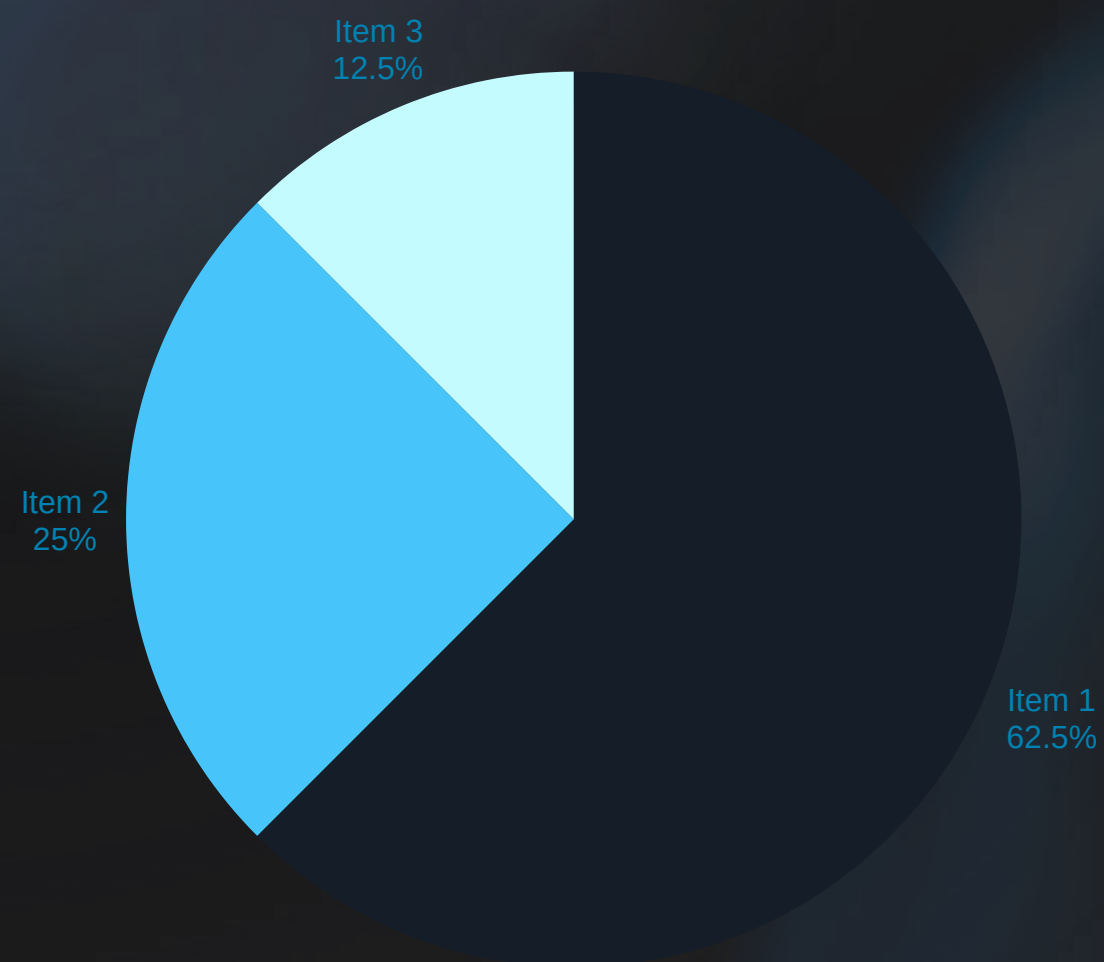
Repairs incomplete outcomes automatically

**Invariant Verification**

Confirms correctness & customer status

How Do We Monitor & Recover?

The operational loop closes the reliability circle: telemetry streams from every component feed anomaly detection; alerts trigger automated runbooks; recovery workers repair incomplete transactions; operators verify invariants hold and customer-visible status is accurate.



The Architecture Behind the Answers

Complete request flow: Edge admission control filters traffic → API gateway enforces rate limits → Authoritative inventory store processes atomic reservations → Order service orchestrates workflow → Payment provider confirms funds → Message broker queues async work → Workers handle fulfillment → Reconciliation validates consistency → Telemetry feeds operational dashboards.

1

single source of truth

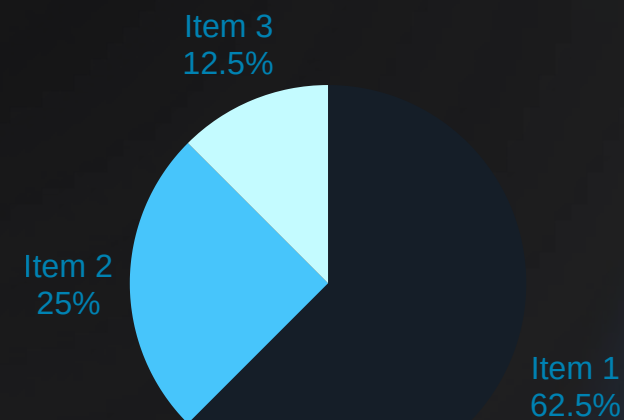
∞

bounded async work

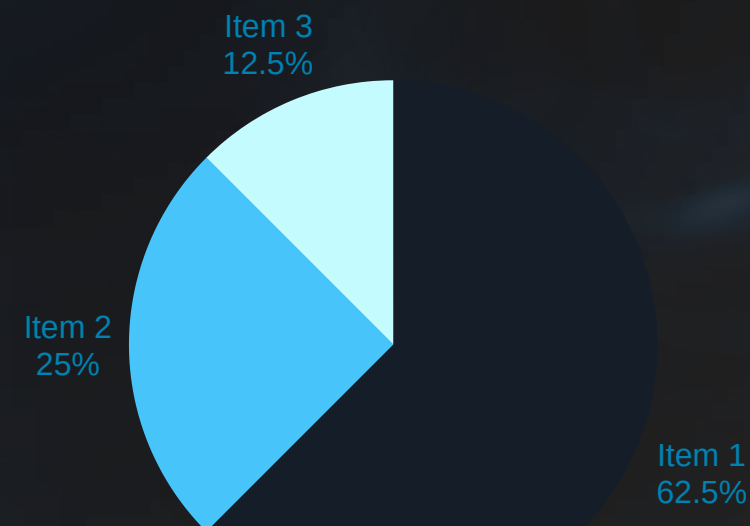
Proof, Not Promises

Every guarantee is validated through rigorous testing: concurrency race simulations, invariant verification, duplicate-request detection, payment crash injection, expiry and late-payment scenarios, message broker replay testing, and 50× load stress tests. Evidence targets: zero oversells, exactly-once logical purchase outcomes, bounded recovery time under 5 seconds, and graceful rejection with clear feedback under saturation conditions.

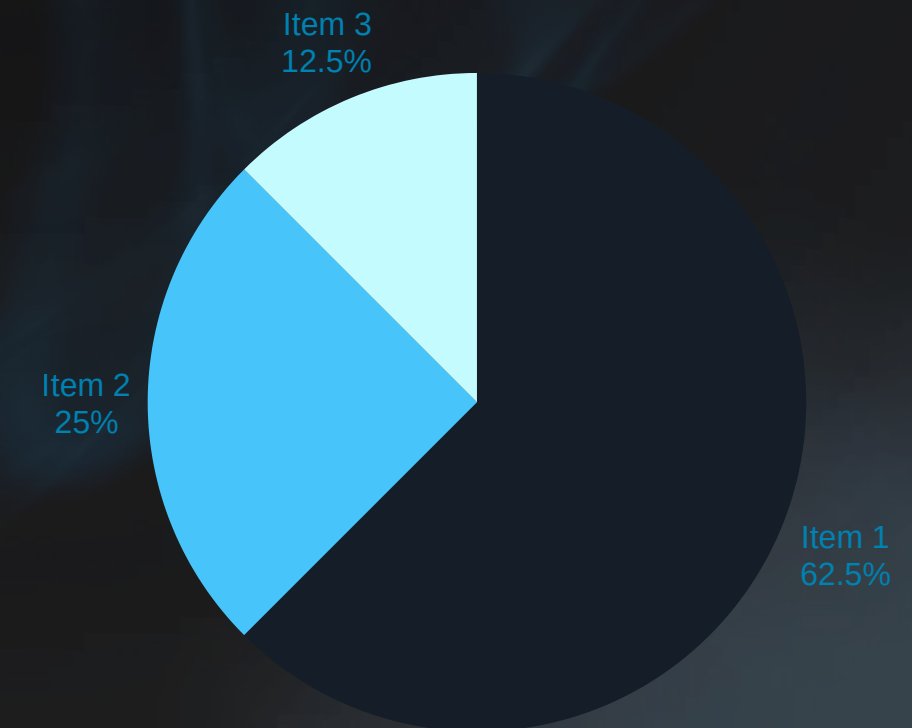
Test Coverage by Category



Load Test Results (Requests/sec)



Oversell Rate Across Tests



SALESTORM

Built Around Correctness

Fast admission · Authoritative reservations ·
Recoverable payment · Graceful overload



[GITHUB](#)